

EE7207 Neural Networks & Deep Learning

From Tokenization to Transformer & BERT

Jiawei Weng

March 20, 2026



Group: EE7207 Peer Tutoring



Valid until 4/3 and will update upon joining group

Scan to join Wechat Group

Jiawei Weng

Peer Tutoring Attendance form



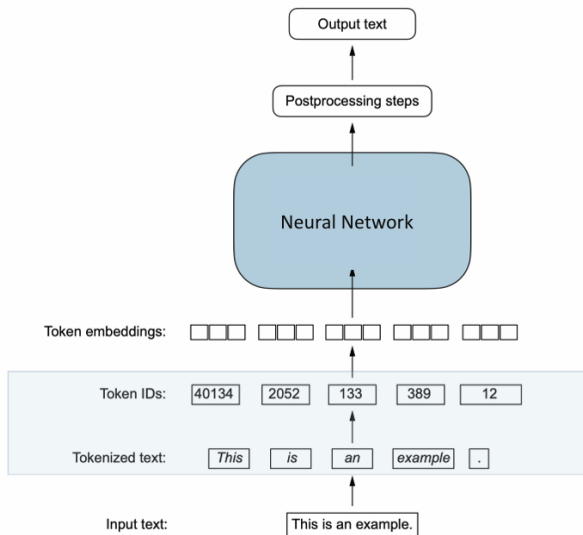
Scan to take attendance

1.1 Tokenization: Breaking Down the Language

1. The Concept

- **Definition:** Splitting raw text into smaller units called **Tokens**.
- **The Dilemma:** Word-based (too large vocab, OOV issues) vs. Character-based (too long, loses meaning).

1.1 Tokenization: Breaking Down the Language



1.1 Tokenization: Breaking Down the Language

Byte Pair Encoding (BPE)

- **Mechanism:** Iteratively merge the most frequent adjacent character pairs.
- **First Principle:** Use statistics to find the most efficient sub-word units.
- **Benefit:** Efficiently handles "Out-of-Vocabulary" (OOV) by falling back to sub-units (e.g., un + break + able).

1.2 Byte Pair Encoding (BPE)

1. Initialization:

- Start with a corpus where each word is split into characters and appended with a special end-of-word marker (e.g., $w \rightarrow w_$).

2. Count Character Pair Frequencies:

- Count how often each adjacent pair of characters appears in the corpus.

3. Merge Most Frequent Pair:

- Merge the most frequent pair into a single unit.

4. Repeat Iteratively:

- Continue merging until a predefined number of merges is reached or vocabulary size is sufficient.

5. Encoding Unknown Words:

- For unseen words, BPE splits them into known subwords.

Exam Case 1: 2023-2024-S2-Q3(c): BPE Derivation

3. (a) Consider the sentence: "fred fed ted bread". Perform Byte Pair Encoding (BPE) step-by-step. Only perform 2 merges. At each merge:
- (i) Identify the most frequent pair.
 - (ii) Merge the pair in all occurrences.
 - (iii) Update your vocabulary.

Your answer should clearly demonstrate each intermediate step.

(8 Marks)

Step 0: Initialization

- Sequences: f r e d, f e d, t e d, b r e a d
- Initial Vocab: {a, b, d, e, f, r, t}

Step 1: First Merge

- **Identify:** (e, d) appears 3 times (in *fred*, *fed*, *ted*).
- **Merge:** f r ed, f ed, t ed, b r e a d.
- **Update Vocab:** {a, b, d, e, f, r, t, ed}.

Step 2: Second Merge

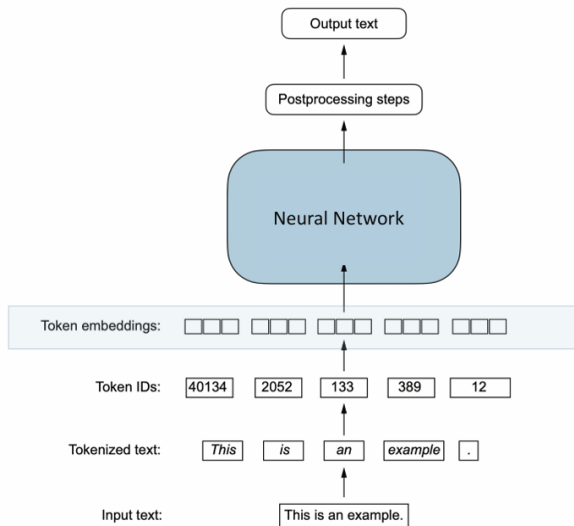
- **Identify:** All remaining pairs have frequency 1. Picking **(r, ed)** from *f r ed*.
- **Merge:** *f red, f ed, t ed, b r e a d*.
- **Update Vocab:** {a, b, d, e, f, r, t, ed, red}.

2. Word Embedding: From Discrete to Continuous

1. Core Concept

- A mapping that converts discrete **Token IDs** into **low-dimensional, dense, continuous** vectors.
- **First Principle:** Geometric distance represents semantic similarity.

2. Word Embedding: From Discrete to Continuous



2. Word Embedding: From Discrete to Continuous

The Lookup Table Mechanism

- An Embedding layer is a matrix $W \in \mathbb{R}^{V \times d}$ (V : Vocab size, d : Embedding dim).
- **Operation**: Fetching the i -th row of W using Token ID i .
- **Learning**: Values are not fixed; they are **learnable parameters** updated via backpropagation.

Focus: Why Use Embeddings?

Feature	One-Hot Encoding	Word Embedding
Dimension	High-dim (V , e.g., 50k)	Low-dim (d , e.g., 768)
Sparsity	Sparse (mostly zeros)	Dense (real numbers)
Semantics	None (Orthogonal)	High (Captures similarity)
Calculation	Matrix multiplication (slow)	Index lookup (fast)

Exam Tip: Semantic Similarity

In the embedding space, $\text{vec}(\text{Apple}) \cdot \text{vec}(\text{Pear})$ is high, while $\text{vec}(\text{Apple}) \cdot \text{vec}(\text{Car})$ is low. It captures analogies like: *King - Man + Woman $\approx$ Queen*.

3. Self-Attention: The "Search" Mechanism

1. The Scaled Dot-Product Attention

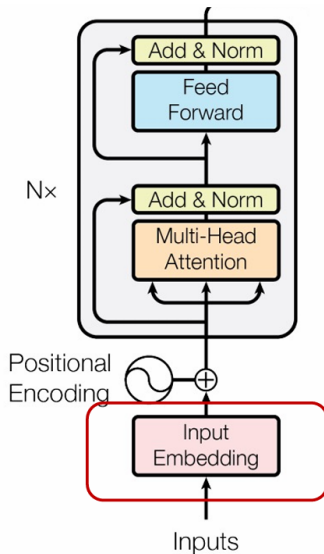
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- **Q (Query)**: What I'm looking for.
- **K (Key)**: What I offer.
- **V (Value)**: The content I carry.

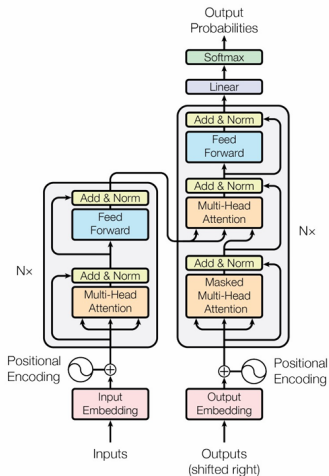
2. Why Scale by $\sqrt{d_k}$?

- Prevents the dot product from exploding, which would push the **softmax into saturation** (vanishing gradients).

4. Transformer Architecture: Encoder



4. Transformer Architecture: Encoder + Decoder



4. Transformer: Parallelism and Stability

1. Positional Encoding

- Attention is "bag-of-words" (orderless). We must inject position signals to maintain sequence info.

2. Multi-Head Attention

- Allows the model to attend to information from different representation subspaces simultaneously.

3. Add & Norm (Residuals + LayerNorm)

- **Residuals:** Allow gradients to flow through deep layers without vanishing.
- **LayerNorm:** Stabilizes the hidden states within each layer.

- (c) Draw the encoder architecture of the typical transformer model. Explain the difference between BERT and GPT in terms of their model architectures. Give two examples of common use cases for BERT and GPT, respectively.

(6 Marks)

Question (c-ii): Explain the difference between BERT and GPT in terms of their model architectures.

- **BERT (Encoder-based):**

- Uses **Bidirectional** self-attention.
- Can see both left and right context at any position.
- Designed for NLU (Natural Language Understanding).

- **GPT (Decoder-based):**

- Uses **Unidirectional** (Masked) self-attention.
- Each token can only attend to previous tokens.
- Designed for NLG (Natural Language Generation).

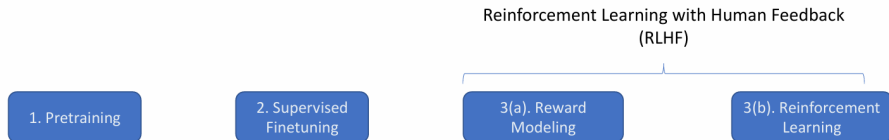
Question (c-iii): Give two common use cases for BERT and GPT, respectively. 1. BERT (Understanding Tasks):

- **Sentiment Analysis:** Determining the emotional tone of text.
- **Named Entity Recognition (NER):** Identifying names, dates, or locations.

2. GPT (Generative Tasks):

- **Text Completion:** Automatically finishing a sentence or story.
- **Dialogue Systems:** Powering chatbots and virtual assistants.

Training Pipeline for GPT/LLM



5 Training Pipeline for GPT/LLM

- 1 **Pretraining:** Train a *Base Model* on trillions of tokens (e.g., FineWeb) to predict the next token.
- 2 **Supervised Finetuning (SFT):** Use high-quality, human-written instruction-answer pairs to create an *Assistant Model*.
- 3 **RLHF:** Align the model with human preferences using **Reward Modeling** and **Reinforcement Learning** (e.g., PPO).

5 Step 1 - Pretraining Deep Dive

Dataset

Internet text: trillions of words, low quality, large quantity

Task

Language modelling: predict the next word

Output

Base model (foundation model)

Notes

1000s GPUs, months of training, \$100M to train GPT-4

5 Step 1 - Pretraining Deep Dive

Key Characteristics of Pretraining:

- **Data Scale:** Trillions of tokens (e.g., 15 Trillion for FineWeb).
- **Objective:** Causal Language Modeling (Next Token Prediction).
- **Processing:** Requires **Deduplication** and **Quality Filtering** (e.g., removing toxic or low-quality web content).
- **Hardware:** Massive GPU clusters (A100/H100) and months of compute time.

Result: A **Base Model** that is a "document completer" but not yet a conversational assistant.

5 Step 2 - Supervised Finetuning (SFT)

Dataset

Sample Question-Answer pairs, ideal assistant responses, 10-100k pairs, high quality

Task

Language modelling: predict the next word

Output

SFT model

Notes

Init from base model, 1-100 GPUs, days of training, less than 1% of pretraining compute

5 Step 2 - Supervised Finetuning (SFT)

Objective: Transform a *Base Model* into an *Assistant Model* by learning to follow human instructions.

- **Data Collection:**

- Small but high-quality dataset (e.g., 10k - 100k examples).
- Consists of (Prompt, Response) pairs written by human AI tutors.

- **Training Process:**

- Perform standard Language Modeling (predict next token) on the **human-written responses** only.

Key Change: The model stops being a mere "document completer" and starts behaving like a **Conversational Assistant**.

5 SFT: Why is it not enough?

Limitations of Stage 2 (SFT):

- **Hallucination:** The model might still generate plausible-sounding but factually incorrect information.
- **Alignment Gap:** It's difficult and expensive to write "perfect" answers for every possible creative or complex prompt.
- **Safety:** SFT alone might not sufficiently catch all edge cases for harmful content.

Transition: This leads to **Stage 3 (RLHF)** to further refine the model's behavior based on human preferences.

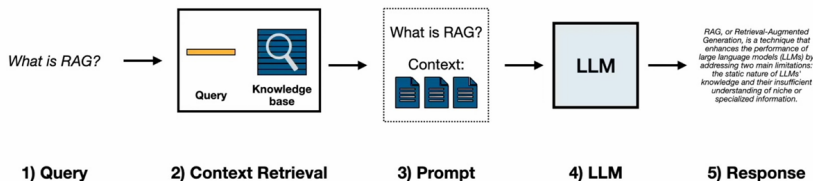
Question: Why is RLHF necessary after Supervised Finetuning?

- **Human Alignment:** To ensure the model is helpful, honest, and harmless (3Hs).
- **Comparison over Generation:** It is easier for humans to *rank* multiple answers than to *write* a perfect one.
- **Two-Stage Process:**
 - **3(a) Reward Model:** Trained on human-ranked comparisons to predict which answer is better.
 - **3(b) Policy Optimization:** Use RL (PPO) to update the LLM to maximize the Reward Model's score.

- (b) Imagine you want to train a GPT-like assistant. The training process can be broken down into multiple steps.
- (i) List each of these training steps in the correct sequence.
 - (ii) For each step, explain the dataset used, the training task, and the output of this step (what model is produced).

(8 Marks)

6 RAG



Limitations of LLMs:

- **Knowledge Cutoff:** Models cannot access information after their training date.
- **Hallucination:** Models may generate factually incorrect but plausible-sounding text.
- **Domain Specialization:** Lack of access to private or niche documents.

The RAG Solution:

- **Retriever:** Identify the most relevant chunks from a knowledge base (Vector DB).
- **Generator:** Produce an answer conditioned on both the query and the retrieved context.

Two Stages of Evaluation:

- **Retriever Evaluation:** Focus on finding relevant documents.
 - Metrics: Recall@k, Precision@k, MRR, MAP.
- **Generator Evaluation:** Focus on text accuracy and fluency.
 - Metrics: **ROUGE**, BLEU, BERTScore.

ROUGE Definition

Recall-Oriented Understudy for Gisting Evaluation. It measures how much of the *reference* summary is captured by the *generated* summary.

Exam Case: RAG 2024-2025-S2-Q3(c)

- (c) You are given the following reference summary (R) and generated summary (H):

Reference (R):

“large language models power modern NLP systems with impressive generative capabilities”

Generated (H):

“modern NLP uses large language models for generative tasks”

Answer the following:

- (i) List all bigrams from each summary (assume tokenization by whitespace).
- (ii) Identify the overlapping bigrams between R and H.
- (iii) Compute ROUGE-2 Recall, ROUGE-2 Precision, and ROUGE-2 F1.

(9 Marks)

Exam Case: ROUGE-2 Calculation (i & ii)

Reference (R): “large language models power modern NLP systems with impressive generative capabilities”

Generated (H): “modern NLP uses large language models for generative tasks”

- **(i) List Bigrams (Whitespace Tokenization):**

- **R (10 bigrams):** {large language, language models, models power, power modern, modern NLP, NLP systems, systems with, with impressive, impressive generative, generative capabilities}
- **H (8 bigrams):** {modern NLP, NLP uses, uses large, large language, language models, models for, for generative, generative tasks}

- **(ii) Overlapping Bigrams:**

- ① “modern NLP”
- ② “large language”
- ③ “language models”

Count = 3 overlapping bigrams.

Exam Case: ROUGE-2 Calculation (iii)

Computing Scores:

- **ROUGE-2 Recall:** $\frac{\text{Overlap}}{\text{Total bigrams in R}} = \frac{3}{10} = \mathbf{0.3}$
- **ROUGE-2 Precision:** $\frac{\text{Overlap}}{\text{Total bigrams in H}} = \frac{3}{8} = \mathbf{0.375}$
- **ROUGE-2 F1:** $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
 - $F1 = 2 \times \frac{0.375 \times 0.3}{0.375 + 0.3} = 2 \times \frac{0.1125}{0.675} = \frac{0.225}{0.675} \approx \mathbf{0.333}$

Note: Recall emphasizes completeness of the reference, while Precision emphasizes the quality of the hypothesis.

Definition: A system that uses an LLM as its *reasoning engine*, enhanced with planning, memory, and tool usage.

- **LLM Core:** The central brain for understanding and generating.
- **Planning:** Breaking down goals into actionable steps.
- **Memory:** Retaining context across short-term or long-term interactions.
- **Tools:** External APIs (Search, Calculator, Code Runner) to extend capabilities.

7 Planning Reasoning Strategies

- **Chain-of-Thought (CoT):** Instructs the model to generate intermediate reasoning steps (“thinking out loud”) before the final answer.
- **Hierarchical Planning:** Decomposing a high-level goal into sub-goals and atomic tasks (e.g., Writing a research paper).
- **ReACT (Reason + Act):** An iterative loop where the agent alternates between *reasoning* and *taking action* (calling tools).

ReACT Workflow

Think → Act (Tool Call) → Observe (Result) → Refined Thought.

7 Agent Memory Mechanisms

- **Short-Term Memory:** Within the *Context Window*. Stores ongoing conversation and immediate sub-goals.
- **Long-Term Memory:** Beyond the context window. Uses an *external store* (Vector DB) via RAG to recall past sessions or user preferences.

7 Multi-Agent Systems (MAS)

Core Idea: Multiple agents with distinct skill sets (e.g., Coder, Manager) working together.

- **Benefits:** Parallelism (speed) and Specialization (expertise).
- **Coordination:**
 - **Centralized:** A “manager” agent delegates tasks.
 - **Distributed:** Agents share results peer-to-peer.

7 Autonomous Agents vs. Agentic Workflow

Feature	Autonomous Agents	Agentic Workflow
<i>Nature</i>	Thinking for themselves under uncertainty.	Strict, pre-scripted sequence of steps.
<i>Pros</i>	Highly adaptable and innovative.	Predictable, reliable, and easy to debug.
<i>Cons</i>	Unpredictable behavior.	Inflexible to novel situations.

EE7207 LLM Knowledge Map

